Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

Department of CSE

B.Tech. CSE Trimester -5

Roll Number:2520030609
Name:V.Siri Chandana

Generative AI and AI Platforms

25CS2207E

AIM

To develop an LLM Monitoring System using Python, Ollama, and MongoDB to generate AI responses and store LLM usage details in MongoDB.

SOURCE CODE

1. cost_calculator.py

```python
# cost_calculator.py

# Example rates per 1 million tokens.
# IMPORTANT: These are sample values, not verified current Gemini prices.
# Replace them with the actual rates for your selected model.

PRICING = {
    "gemini-3.8-flash": {
        "input": 0.50,
        "output": 2.00
    }
}


def calculate_cost(provider, model, input_tokens, output_tokens):

    # Ollama runs locally and does not charge a per-token API fee.
    if provider == "OllamaProvider":
        return 0.0

    rates = PRICING.get(model)

    if rates is None:
        return None

    input_cost = (
        input_tokens * rates["input"]
    ) / 1_000_000

    output_cost = (
        output_tokens * rates["output"]
    ) / 1_000_000

    total_cost = input_cost + output_cost

    return round(total_cost, 10)
```

2. llm_client.py

```
# llm_client.py

from abc import ABC, abstractmethod
from google import genai
from google.genai import types
import requests
import time

from cost_calculator import calculate_cost



# ----------------------------------------------------------------------
# 1. Common LLM Provider Interface
# ----------------------------------------------------------------------

class LLMProvider(ABC):

    @abstractmethod
    def generate(self, prompt):
        pass



# ----------------------------------------------------------------------
# 2. Gemini Provider
```

```
# ----------------------------------------------------------------------


class GeminiProvider(LLMProvider):


    def __init__(self, api_key):


        self.client = genai.Client(

            api_key=api_key,

            http_options=types.HttpOptions(timeout=30000)

        )


        self.model = "gemini-3-flash-pro"


    def generate(self, prompt):


        response = self.client.models.generate_content(

            model=self.model,

            contents=prompt

        )


        usage = response.usage_metadata


        input_tokens = (

            usage.prompt_token_count if usage else 0

        )
```

```
        output_tokens = (
            usage.candidates_token_count if usage else 0
        )

        return {
            "response": response.text,
            "input_tokens": input_tokens,
            "output_tokens": output_tokens
        }



# ------------------------------------------------------------------
# 3. Ollama Provider
# ------------------------------------------------------------------

class OllamaProvider(LLMProvider):

    def __init__(self, model="llama3.2"):

        self.model = model

        self.url = "http://localhost:11434/api/generate"

    def generate(self, prompt):

        response = requests.post(
```

```
            self.url,
            json={
                "model": self.model,
                "prompt": prompt,
                "stream": False
            },
            timeout=30
        )

        response.raise_for_status()

        data = response.json()

        return {
            "response": data["response"],
            "input_tokens": data.get("prompt_eval_count", 0),
            "output_tokens": data.get("eval_count", 0)
        }



# ----------------------------------------------------------------------
# 4. Common LLM Client
# ----------------------------------------------------------------------

class LLMClient:
```

```
def __init__(self, provider, logger):


    self.provider = provider

    self.logger = logger


def generate(self, prompt):


    max_attempts = 3


    start_time = time.perf_counter()


    last_error = None


    for attempt in range(1, max_attempts + 1):


        try:


            # Generate AI response

            result = self.provider.generate(prompt)


            # Calculate total elapsed time, including retries

            latency = round(

                (time.perf_counter() - start_time) * 1000,

                2

            )
```

```
            input_tokens = result["input_tokens"]
            output_tokens = result["output_tokens"]

            model = self.provider.model

            provider_name = type(self.provider).__name__

            cost = calculate_cost(
                provider_name,
                model,
                input_tokens,
                output_tokens
            )

            # Prepare MongoDB document
            log_data = {
                "provider": provider_name,
                "model": model,
                "prompt": prompt,
                "input_tokens": input_tokens,
                "output_tokens": output_tokens,
                "total_tokens": input_tokens + output_tokens,
                "latency_ms": latency,
                "cost": cost,
                "attempts": attempt,
                "status": "success",
```

```
                "error": None
            }

            # Log separately: a MongoDB error must not repeat
            # an already successful model request
            try:
                self.logger.log_call(log_data)
            except Exception as log_error:
                print("MongoDB logging error:", log_error)

            return result["response"]

        except Exception as error:

            last_error = error

            # Retry only likely temporary failures
            status_code = getattr(error, "status_code", None)

            retryable = (
                isinstance(
                    error,
                    (
                        requests.exceptions.Timeout,
                        requests.exceptions.ConnectionError
                    )
```

```
            )
            or status_code == 429
            or (
                isinstance(status_code, int)
                and status_code >= 500
            )
            or (
                isinstance(error, requests.exceptions.HTTPError)
                and error.response is not None
                and (
                    error.response.status_code == 429
                    or error.response.status_code >= 500
                )
            )
        )

        if not retryable or attempt == max_attempts:
            break

        wait_time = 2 ** (attempt - 1)

        print(
            f"Attempt {attempt} failed. "
            f"Retrying in {wait_time} seconds..."
        )
```

```
            time.sleep(wait_time)


    # All attempts failed, or the error was not retryable
    latency = round(
        (time.perf_counter() - start_time) * 1000,
        2
    )

    failed_log = {
        "provider": type(self.provider).__name__,
        "model": self.provider.model,
        "prompt": prompt,
        "latency_ms": latency,
        "attempts": attempt,
        "status": "failed",
        "error": str(last_error)
    }

    try:
        self.logger.log_call(failed_log)
    except Exception as log_error:
        print("MongoDB logging error:", log_error)

    raise last_error
```

3. mongo_logger.py

```
# mongo_logger.py

from pymongo import MongoClient
from datetime import datetime, timezone


class MongoLogger:

   def __init__(self):

       self.client = MongoClient(
           "mongodb://localhost:27017/"
       )

       self.db = self.client["llm_monitoring"]

       self.collection = self.db["call_logs"]

   def log_call(self, log_data):

       log_data["timestamp"] = datetime.now(timezone.utc)

       self.collection.insert_one(log_data)

       print("Call details saved in MongoDB!")

   def close(self):

       self.client.close()
```

4. main.py

```
# main.py


import os


from dotenv import load_dotenv
```

```
from llm_client import (
    GeminiProvider,
    OllamaProvider,
    LLMClient
)

from mongo_logger import MongoLogger



# Load environment variables
load_dotenv(override=True)



print("================================")
print("     WEEK 3: LLM MONITORING")
print("================================")

print("1. Gemini")
print("2. Ollama")

choice = input("\nSelect provider (1/2): ").strip()

logger = None

try:
```

```
# Connect to MongoDB
logger = MongoLogger()


# Select AI provider
if choice == "1":


    api_key = os.getenv("GEMINI_API_KEY")


    if not api_key:
        raise ValueError("Gemini API key not found in .env")


    provider = GeminiProvider(api_key)


elif choice == "2":


    provider = OllamaProvider("llama3.2")


else:


    raise ValueError("Invalid selection. Choose 1 or 2.")


# Create common client
client = LLMClient(provider, logger)


# Get user prompt
prompt = input("\nEnter your question: ")
```

```
    # Generate response
    answer = client.generate(prompt)

    print("\n========== AI RESPONSE ==========")
    print(answer)

    print("\n========== COMPLETED ==========")

except Exception as error:

    print("\nApplication error:", error)

finally:

    if logger is not None:
        logger.close()
```

5. requirements.txt

```
pytest
google-genai
python-dotenv
pymongo
```

OUTPUT

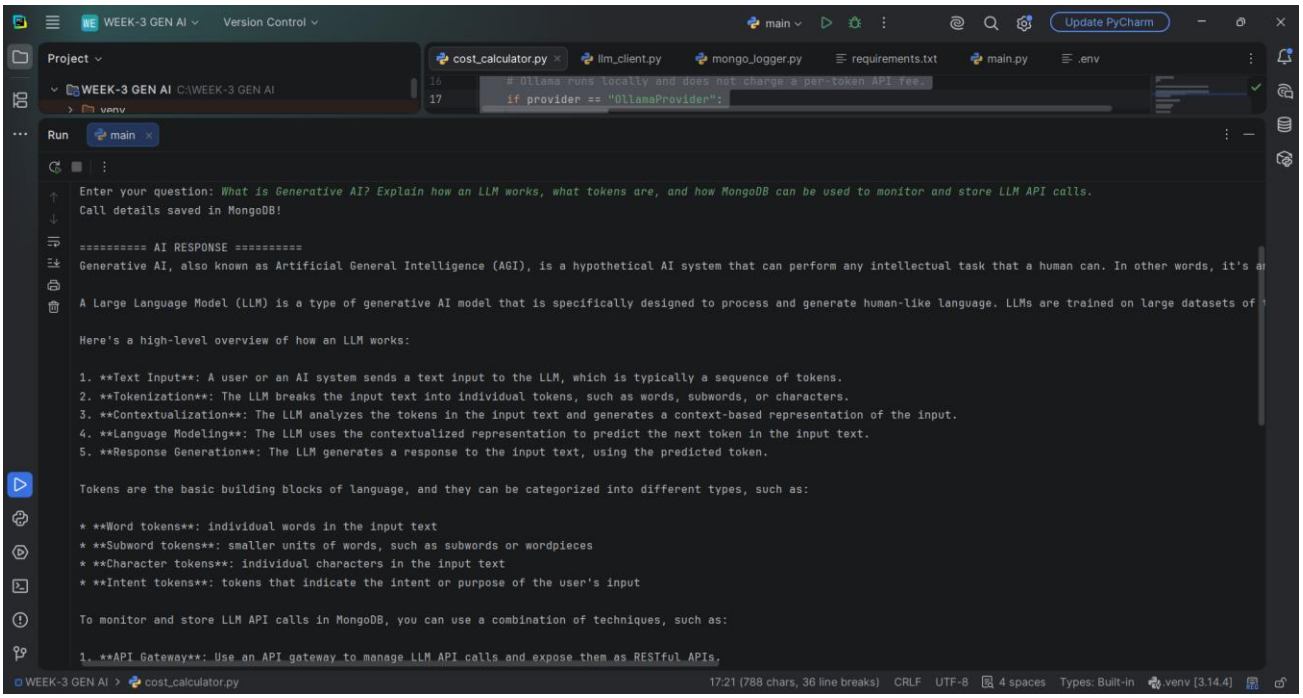
Output 1: MongoDB Service Verification

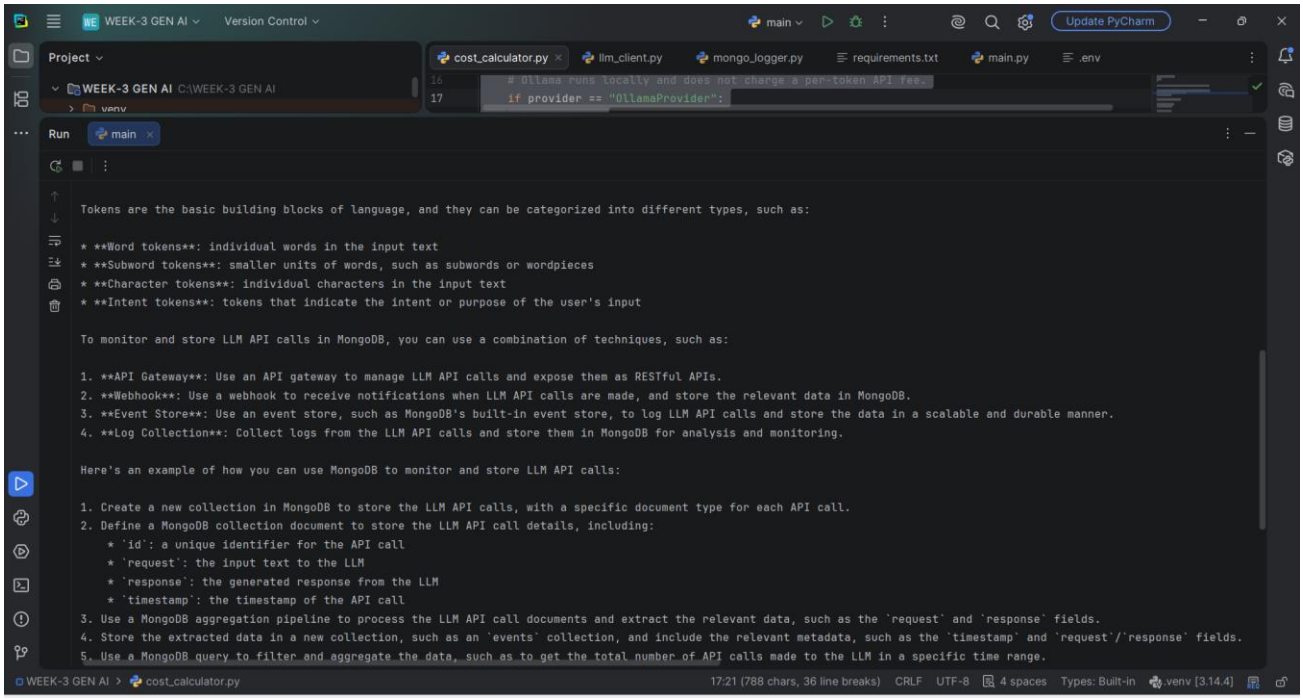
```
(.venv) PS C:\WEEK-3 GEN AI> Get-Service MongoDB

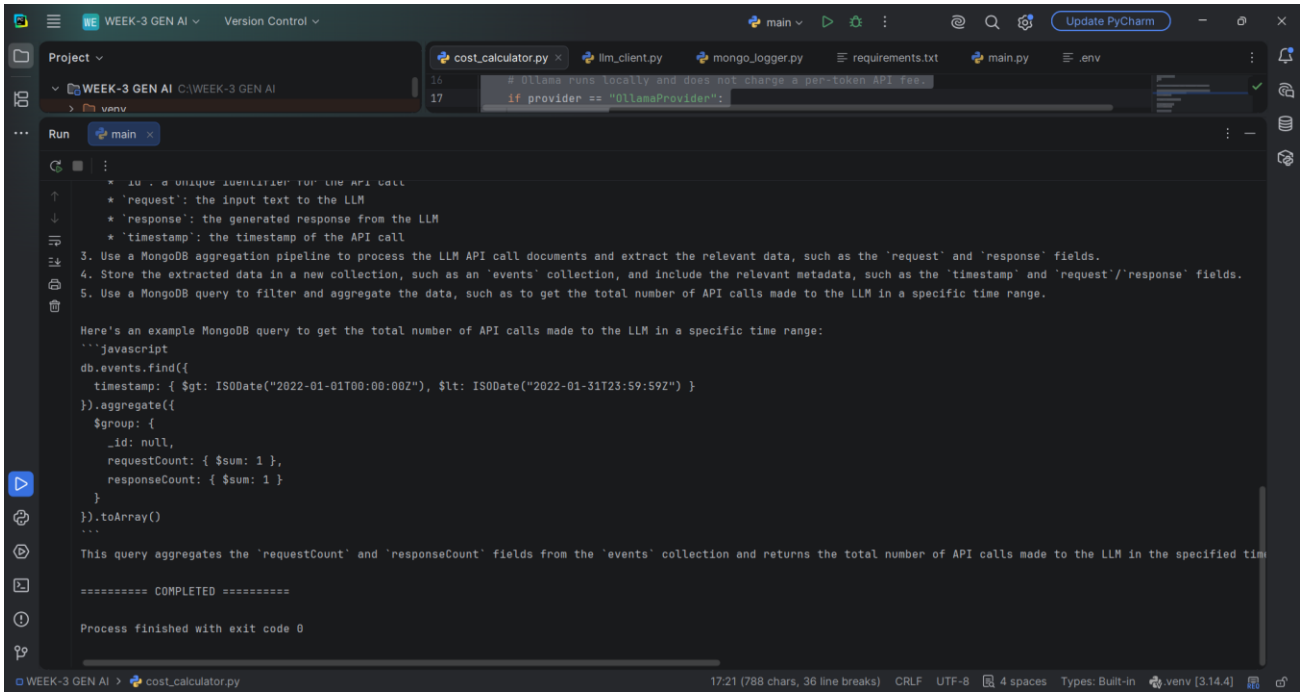
Status   Name               DisplayName
------   ----               -----------
Running  MongoDB            MongoDB Server (MongoDB)
```

Output: MongoDB service is Running.

Output 2: Program Execution




WEEK-3 GEN AI
Version Control
main
Update PyCharm
Project
WEEK-3 GEN AI C:\WEEK-3 GEN AI
venv
cost_calculator.py
llm_client.py
mongo_logger.py
requirements.txt
main.py
.env
Ollama runs locally and does not charge a per-token API fee.
if provider == "OllamaProvider":
Run
main
Tokens are the basic building blocks of language, and they can be categorized into different types, such as:
* **Word tokens**: individual words in the input text
* **Subword tokens**: smaller units of words, such as subwords or wordpieces
* **Character tokens**: individual characters in the input text
* **Intent tokens**: tokens that indicate the intent or purpose of the user's input
To monitor and store LLM API calls in MongoDB, you can use a combination of techniques, such as:
1. **API Gateway**: Use an API gateway to manage LLM API calls and expose them as RESTful APIs.
2. **Webhook**: Use a webhook to receive notifications when LLM API calls are made, and store the relevant data in MongoDB.
3. **Event Store**: Use an event store, such as MongoDB's built-in event store, to log LLM API calls and store the data in a scalable and durable manner.
4. **Log Collection**: Collect logs from the LLM API calls and store them in MongoDB for analysis and monitoring.
Here's an example of how you can use MongoDB to monitor and store LLM API calls:
1. Create a new collection in MongoDB to store the LLM API calls, with a specific document type for each API call.
2. Define a MongoDB collection document to store the LLM API call details, including:
* `id`: a unique identifier for the API call
* `request`: the input text to the LLM
* `response`: the generated response from the LLM
* `timestamp`: the timestamp of the API call
3. Use a MongoDB aggregation pipeline to process the LLM API call documents and extract the relevant data, such as the `request` and `response` fields.
4. Store the extracted data in a new collection, such as an `events` collection, and include the relevant metadata, such as the `timestamp` and `request`/`response` fields.
5. Use a MongoDB query to filter and aggregate the data, such as to get the total number of API calls made to the LLM in a specific time range.
WEEK-3 GEN AI > cost_calculator.py
17:21 (788 chars, 36 line breaks) CRLF UTF-8 4 spaces Types: Built-in .venv [3.14.4]


WEEK-3 GEN AI
Version Control
main
Update PyCharm
Project
WEEK-3 GEN AI C:\WEEK-3 GEN AI
venv
cost_calculator.py
llm_client.py
mongo_logger.py
requirements.txt
main.py
.env
Ollama runs locally and does not charge a per-token API fee.
if provider == "OllamaProvider":
Run
main
* `request`: the input text to the LLM
* `response`: the generated response from the LLM
* `timestamp`: the timestamp of the API call
3. Use a MongoDB aggregation pipeline to process the LLM API call documents and extract the relevant data, such as the `request` and `response` fields.
4. Store the extracted data in a new collection, such as an `events` collection, and include the relevant metadata, such as the `timestamp` and `request`/`response` fields.
5. Use a MongoDB query to filter and aggregate the data, such as to get the total number of API calls made to the LLM in a specific time range.
Here's an example MongoDB query to get the total number of API calls made to the LLM in a specific time range:
```javascript
db.events.find({
timestamp: { $gt: ISODate("2022-01-01T00:00:00Z"), $lt: ISODate("2022-01-31T23:59:59Z") }
}).aggregate({
$group: {
_id: null,
requestCount: { $sum: 1 },
responseCount: { $sum: 1 }
}
}).toArray()
```
This query aggregates the `requestCount` and `responseCount` fields from the `events` collection and returns the total number of API calls made to the LLM in the specified tim
========== COMPLETED ==========
Process finished with exit code 0
WEEK-3 GEN AI > cost_calculator.py
17:21 (788 chars, 36 line breaks) CRLF UTF-8 4 spaces Types: Built-in .venv [3.14.4]

```
_id: ObjectId('6af43a874cdbbea79843f80e')
provider : "OllamaProvider"
model : "llama3.2:1b"
prompt : "What is Generative AI?"
input_tokens : 31
output_tokens : 623
total_tokens : 654
latency_ms : 32874.49
cost : 0
attempts : 1
status : "success"
error : null
timestamp : ISODate('2026-11-11T09:51:03.165+00:00') November 11, 2026 at 9:51:03 AM UTC
```

```
_id: ObjectId('6af43ae92969ff23fae87c3d')
provider : "OllamaProvider"
model : "llama3.2:1b"
prompt : "What is Generative AI? Explain how an LLM works, what tokens are, and …"
input_tokens : 57
output_tokens : 865
total_tokens : 922
latency_ms : 36618.89
cost : 0
attempts : 1
status : "success"
error : null
timestamp : ISODate('2026-11-11T09:52:41.740+00:00') November 11, 2026 at 9:52:41 AM UTC
```